

The Ghost in the Cache: Recovering Deleted AI Chat Histories from Chromium LevelDB

Paper #2060 — Anonymous Submission

Abstract

Users increasingly trust web-based Large Language Model (LLM) interfaces—ChatGPT, Claude, Gemini, DeepSeek—with sensitive data such as credentials, API keys, and proprietary code, assuming that deleting a conversation erases it from their machine. We show this assumption is false. Chromium-based browsers persist LLM conversation data in LevelDB-backed IndexedDB storage, and the log-structured merge-tree (LSM-tree) architecture retains “deleted” records as tombstone entries until background compaction physically reclaims the underlying blocks—a process yielding recoverable artifacts with an observed storage age of up to 83 days on our test workstation in a low-activity profile.

We present TINKER TAILOR, a pure-Python forensic carving tool that recovers deleted and active LLM chat histories from live browser sessions without compiled dependencies or database locks. The tool reverse-engineers Chromium’s V8 structured-clone serialization format—Smi-shifted array indices, OneByteString (Latin-1) and TwoByteString (UTF-16LE) encoding, and nested-object depth tracking—and implements a zero-dependency Snappy decompressor for compressed SSTables. For Claude, it additionally recovers *unsubmitted* prompt drafts from cached TipTap/ProseMirror editor state, capturing text the user typed but never sent.

In a single-workstation case study spanning 23 Chrome profiles, TINKER TAILOR recovers 506 text artifacts—204 user prompts, 154 assistant responses, and 148 Claude keystroke fragments—of which **358 (70.8%) originated from conversations the user had deleted** through the application UI. On ChatGPT data, both grep and JSON baselines recover **0%** of what the V8-aware parser finds. Among the Claude artifacts, 30 are substantial draft-level prompts, while 148 are shorter keystroke fragments produced by TipTap’s per-keystroke caching. Crucially, we demonstrate that conversation data *survives* Chrome’s “Clear browsing data” operation—remaining recoverable from residual SSTable files days after the user explicitly requested deletion—and that uploaded documents (e.g., a 10MB PDF) persist as unencrypted blobs on disk. The full extraction completes in under 163 ms with

no database locks, disk writes, network activity, or privilege escalation. We treat the recovery counts and persistence windows as a single-machine case study rather than a population estimate, and discuss implications for digital forensics, enterprise data governance, and the emerging threat of AI-targeted infostealers.

1 Introduction

The rapid adoption of web-based LLM interfaces has created a new category of sensitive data that exists entirely outside traditional enterprise data governance. Employees routinely paste API keys, database credentials, proprietary source code, and confidential documents into ChatGPT, Claude, and similar platforms [19]. A 2024 survey by Cyberhaven found that 78% of knowledge workers use AI assistants at least weekly, and 43% have shared company-confidential data with an LLM interface [6]. Samsung banned ChatGPT internally after engineers inadvertently leaked semiconductor source code through the platform [24].

Users operate under what we term the *ephemeral AI fallacy*: the belief that deleting an LLM conversation removes the data from their machine. This belief is reinforced by LLM platform interfaces that show conversations disappearing from the sidebar when deleted. In reality, the data’s journey through the browser storage stack creates multiple persistent copies that survive user-initiated deletion.

When a user interacts with ChatGPT, Claude, or Gemini, the web application stores conversation state in the browser’s IndexedDB API—a client-side database designed for offline-capable web applications. Chromium implements IndexedDB on top of LevelDB [8], a log-structured merge-tree (LSM-tree) key-value store [17]. The LSM-tree architecture is optimized for write throughput: all writes append to sequential logs, and deletions are recorded as *tombstone markers* rather than physical erasure. The original data blocks persist in write-ahead logs (WAL) and Sorted String Tables (SSTables) until background compaction, which may not occur for days or weeks depending on database activity levels.

This architecture creates both a forensic recovery opportunity and a security vulnerability. A single workstation’s Chrome installation may contain months of LLM interaction history across multiple profiles—including conversations the user believed they deleted—all stored in files lacking application-level encryption, readable by any process running as the same user.

The challenge for forensic recovery is that Chromium does not store IndexedDB values as plaintext JSON. Instead, it serializes JavaScript objects using V8’s internal *structured clone* binary format [28, 30]. This format uses type tags, varint-encoded lengths, Smi-shifted integer indices, and two distinct string encodings (OneByteString for Latin-1 content, TwoByteString for UTF-16LE Unicode). Existing forensic tools that rely on string matching or JSON parsing miss the majority of this data entirely.

We structure our investigation around three research questions:

- RQ1** Can LLM conversation data deleted or never submitted through Chromium-based interfaces remain recoverable from client-side storage?
- RQ2** Why do conventional forensic approaches fail to recover this data?
- RQ3** Can a V8-aware, lock-free carver recover the data reliably and quickly enough to create a practical security concern?

This paper makes the following contributions:

1. **V8 Deserialization.** We reverse-engineer Chromium’s V8 structured-clone format for IndexedDB, documenting the tags relevant to LLM carving, varint encoding, Smi-shifted array indices, and nested-object depth tracking (Section 4).
2. **TipTap Draft Recovery.** We discover that Claude caches unsent prompt drafts via TipTap ProseMirror editor state [12, 27], enabling recovery of text the user typed but never submitted (Section 4.4).
3. **Zero-Dependency Forensic Tool.** We implement TINKER TAILOR, a pure-Python carving tool with a custom Snappy decompressor, requiring no compiled libraries, database locks, or installation—deployable directly on incident-response targets (Section 5).
4. **Real-World Evaluation.** We evaluate on a production workstation with 23 Chrome profiles, recovering 506 text artifacts (204 user prompts, 154 assistant responses, and 148 Claude keystroke fragments) that both grep-based and JSON-based forensic baselines miss entirely (Section 6).

5. **Infostealer-Style Collection PoC.** We demonstrate that a 140-line script can collect a machine’s entire LLM history across 23 profiles in under 163 ms with a minimal forensic footprint—no database locks, network calls, file writes, or privilege escalation, using only `open()/read()` on regular files (Section 7).

Scope and Empirical Framing. Our empirical case study uses one primary macOS workstation (23 profiles) with supplementary Windows 11 and synthetic Linux validations. We treat all quantitative recovery counts (506 text artifacts, 83-day observed storage age) strictly as a single-machine case study demonstrating forensic feasibility and magnitude, not a general population estimate. Crucially, the underlying vulnerabilities we uncover—LevelDB LSM-tree compaction deferral and V8 structured-clone binary serialization—are invariant architectural properties of the Chromium engine itself, shared across Chrome, Edge, Brave, Electron applications (e.g., ChatGPT Desktop), and all host operating systems.

2 Background

2.1 LevelDB and the LSM-Tree Architecture

LevelDB [8] implements a log-structured merge-tree (LSM-tree) [17] optimized for write-heavy workloads. Understanding its storage mechanics is essential for forensic recovery because the LSM-tree architecture fundamentally determines *when* deleted data is physically erased.

Write Path. All writes first append to an in-memory *memtable* (a sorted skip list) and simultaneously to a *write-ahead log* (WAL, `.log` file) for crash recovery. When the memtable reaches its size threshold (default 4MB in Chromium), it becomes immutable and is flushed to an on-disk *Sorted String Table* (SSTable) at Level 0. A new empty memtable and WAL are created for subsequent writes.

Compaction. Background compaction merges SSTables from lower levels into higher levels, maintaining sorted order and enforcing size limits per level. Critically, tombstone entries (deletion markers) are *only physically removed* during compaction when LevelDB can confirm that no older version of the key exists at a higher level. Until that compaction occurs, the original data block containing the pre-deletion value remains intact on disk.

SSTable Format. Each SSTable consists of data blocks (prefix-compressed key-value entries, optionally Snappy-compressed), an index block for binary search, and a 48-byte footer with magic number `0x57fb808b247547db`. Our parser uses the footer to locate the index, then walks data blocks to extract values.

Forensic Implications. The LSM-tree design means that a sequence of operations—create conversation, exchange messages, delete conversation—produces the following on-disk artifacts: (1) the original writes in the WAL log, (2) the writes in one or more Level-0 SSTables after memtable flush, (3) a

Table 1: V8 structured clone serialization tags used in LLM conversation data.

Tag	Name	Payload
0x22	OneByteString	varint len + Latin-1 bytes
0x63	TwoByteString	varint len + UTF-16LE bytes
0x49	Smi Integer	varint (value $\times 2$)
0x55	Uint32	varint
0x4e	Double	8 bytes IEEE 754
0x6f	BeginObject	(none)
0x61	BeginArray	varint length
0x7b	EndObject	(none)
0x40	EndArray	(none)
0x30	Null	(none)
0x5f	Undefined	(none)
0x54	True	(none)
0x46	False	(none)

tombstone entry in a newer WAL or SSTable. The original data at positions (1) and (2) remains readable until compaction physically merges and discards it. We observed an artifact with an observed storage age of 83 days remaining recoverable in a low-activity profile; in active profiles, observed persistence was shorter (Section 6.11). A controlled longitudinal study is required to characterize the persistence distribution precisely.

2.2 V8 Structured Clone Serialization

Chromium’s IndexedDB implementation [5] stores JavaScript objects using V8’s internal structured clone serialization [28, 30], a binary format designed for efficient serialization of arbitrary JavaScript values. This is *not* JSON—it is a compact, tag-based binary encoding that preserves JavaScript type semantics including typed arrays, Date objects, RegExp, and circular references.

Table 1 documents the V8 serialization tags relevant to LLM conversation carving. The V8 specification defines additional types (Map, Set, ArrayBuffer, etc.) that we omit as they do not appear in the LLM data we observed.

Integer Encoding. Integers use Protocol Buffers varint encoding (7 bits per byte, MSB continuation flag). V8 additionally encodes small integers (“Smi”) with a left shift: the serialized value is $\text{actual} \times 2$. Our carver reverses this when interpreting message ordering and role assignment.

String Encoding Duality. V8 selects between OneByteString (tag 0x22, Latin-1 / ISO-8859-1) and TwoByteString (tag 0x63, UTF-16LE) based on the string’s content. English-language conversations typically use OneByteString, but conversations containing emoji, CJK characters, or mathematical symbols use TwoByteString. A forensic carver must handle both encodings transparently. Figure 1 shows the binary layout of a typical V8-serialized conversation entry.

V8 Structured Clone Binary Layout



Figure 1: Binary layout of a V8 structured clone serialized conversation. Tags (0x22, 0x61, 0x6f) delimit strings, arrays, and objects. String matching tools cannot find these fields because they are encoded with type tags and varint lengths, not as plaintext JSON.

2.3 LLM Platform IndexedDB Schemas

We analyzed the IndexedDB schemas used by five major LLM platforms through manual inspection of V8-serialized data and comparison against the platforms’ client-side JavaScript source maps.

ChatGPT stores conversations as V8-serialized objects containing a top-level `id` (conversation UUID), `title` (user-visible name), and a `messages` array. Each message object contains `id` (message UUID), `text` (message content), and implicit role assignment through array index parity. The V8 binary anchor for the `messages` array is the byte sequence 0x22 0x08 (OneByteString tag + length 8) followed by the ASCII bytes “messages” and the BeginArray tag 0x61.

Claude (claude.ai) uses the TipTap rich-text editor framework [27], built on ProseMirror [12]. TipTap persists its editor state as a JSON object stored in IndexedDB under the key `tipTapEditorState`. This JSON contains a ProseMirror document tree with `type: "doc"` at the root, `type: "paragraph"` for blocks, and `type: "text"` leaf nodes containing the actual content. Because TipTap saves state on every keystroke, the IndexedDB store accumulates *progressive substring trails* of the user’s typing, even for prompts that were never submitted.

Gemini (gemini.google.com) stores conversation data in V8 format with field names including `content`, `query`, and `response`. The schema has changed across versions, requiring our generic fallback carver.

DeepSeek and Perplexity use standard JSON-in-V8 schemas with `content`, `text`, and `prompt` fields. Our generic regex carver handles these with noise filtering to exclude HTML, JavaScript, and HTTP header fragments.

2.4 The IndexedDB Storage Pipeline

When a user sends a message to ChatGPT, the web application calls `IDBObjectStore.put()` to persist the conversation. Chromium serializes the JavaScript object via V8’s

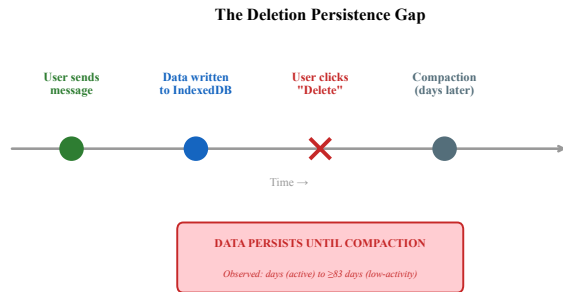


Figure 2: The deletion persistence gap. After the user deletes a conversation, the original data persists in LevelDB SSTables and WAL logs until background compaction physically removes it. In our observations, this ranged from days in active profiles to an observed storage age of 83 days in a low-activity profile.

structured clone serializer and writes the resulting bytes to LevelDB, which appends them to the active WAL and memtable. When the user later deletes the conversation via `IDBObjectStore.delete()`, LevelDB writes a tombstone entry—but the *original data* in the WAL and any flushed SSTables remains physically intact until background compaction. Figure 2 illustrates this deletion persistence gap.

3 Threat Model

We consider three attacker profiles of increasing sophistication, summarized in Table 2. All three require only *filesystem read access* to the browser’s user data directory—no elevated privileges, kernel exploits, or network interception. The **forensic examiner** has physical or logical access to a subject’s machine and needs to recover chat histories as evidence, with chain-of-custody integrity and without modifying the target; this is the primary intended use case for TINKER TAILOR. **Infostealer malware** has achieved unprivileged code execution (e.g., via a phishing document or malicious package) and must exfiltrate before endpoint detection and response (EDR) systems react—a time constraint we estimate at 250–1000 ms across five commercial EDR products from MITRE Engenuity ATT&CK Evaluation data [15]. This threat is acute because LLM conversations aggregate otherwise-scattered sensitive data, the LevelDB directories are user-readable, extraction uses only `open()/read()` on regular files (no API usage EDR heuristics typically monitor), and existing families (Raccoon, Vidar, RedLine) [23] already target browser credential stores. The **insider** has legitimate access to shared workstations or VDI and, where filesystem permissions are not correctly isolated, can read other users’ profile directories—especially relevant in healthcare, legal, and financial settings.

Across macOS, Windows, and Linux, no elevated privileges are required: Chromium’s IndexedDB files sit in user-writable directories readable by any same-user process (per-platform

Table 2: Threat-model matrix. All attacker classes require only user-level filesystem read access to the browser data directory.

Attacker	Recov.?	Elev.?	Primary Defense
Forensic examiner	✓	No	Encryption at rest
Same-user malware	✓	No	EDR + file monitoring
Insider (shared)	✓	No*	Access controls
Offline (FDE)	Depends	—	Full-disk encryption

*Requires access to another user’s home directory (e.g., shared workstation).

paths in Appendix K, Table 18). On macOS, Transparency, Consent, and Control (TCC) may restrict programmatic access in some configurations but does not block same-user access by default.

Our evaluation (Section 6) exercises only the local, full-read scenario common to the forensic-examiner and same-user-malware classes; it runs the tool with the collecting user’s own read access. The insider-on-shared-workstation case—which depends on cross-user permission boundaries on shared machines or VDI—is argued from platform behavior but is *not* empirically tested here, and we flag it as such (Section 8.5).

Our evaluation (Section 6) directly tests the forensic-examiner and same-user-malware scenarios, where the attacker has same-user read access to the browser data directory. The insider-on-shared-workstation scenario is architecturally equivalent (same file access pattern) but was not separately evaluated with a multi-user configuration; we flag this as a limitation.

4 System Design

TINKER TAILOR is structured as a modular carving pipeline with five stages (Figure 3): profile discovery, LevelDB parsing, Snappy decompression, V8 deserialization, and platform-specific carving.

4.1 LevelDB Parser

Our parser processes SSTable files without using the LevelDB native library, avoiding the need for exclusive database locks. The parsing algorithm:

1. Read the last 48 bytes (footer). Validate the 8-byte magic number `0x57fb808b247547db`.
2. Decode two BlockHandle entries from the footer: the meta-index handle and the index handle. Each BlockHandle is a pair of varints (offset, size).
3. Read and optionally decompress the index block.
4. Parse the index block to obtain data block handles.

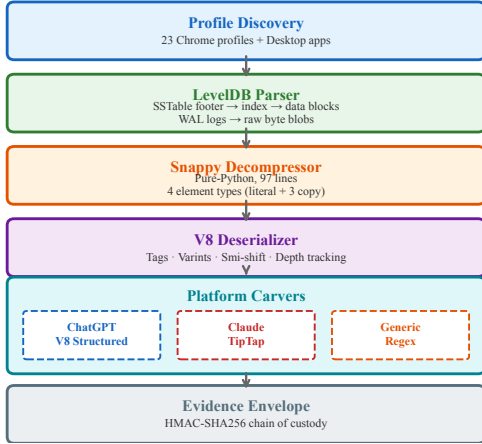


Figure 3: The TINKER TAILOR carving pipeline. Raw LevelDB files are parsed without database locks, decompressed via pure-Python Snappy, deserialized from V8 binary format, and routed through platform-specific carvers.

5. For each data block handle: read the block, check the compression type byte, decompress if Snappy, and parse key-value entries with prefix decompression.

For WAL log files, we read the entire file as a single byte blob and pass it to the V8 carver. While WAL files have a record-level framing format (32KB blocks with CRC-protected records), the V8 carver’s pattern-matching approach handles the interleaved framing bytes without requiring explicit WAL record parsing.

4.2 Pure-Python Snappy Decompressor

SSTables in Chromium’s LevelDB are typically Snappy-compressed. Rather than requiring the `python-snappy` C extension (which depends on the Google Snappy C++ library), we implement a pure-Python Snappy block decompressor in 97 lines.

The Snappy block format [10] consists of:

1. A varint-encoded uncompressed length.
2. A sequence of elements, each beginning with a tag byte where the two least significant bits encode the element type:
 - **0x00: Literal.** The upper 6 bits encode the literal length (minus one) for lengths 1–60. For lengths 61–64, the upper bits indicate 1–4 additional length bytes follow.
 - **0x01: Copy with 1-byte offset.** Length is $4 + ((tag \gg 2) \& 7)$ (range 4–11). Offset is $((tag \& 0xe0) \ll 3) | next\ byte$.
 - **0x02: Copy with 2-byte offset.** Length is $1 + (tag \gg 2)$. Offset is the next 2 bytes (little-endian).

Algorithm 1 V8 Structured Clone Message Carver

Require: Raw LevelDB value bytes V

Ensure: List of $(role, text, id)$ message tuples

```

1:  $anchor \leftarrow 0x22 \backslash x08 messages \backslash x61$ 
2: for each occurrence of  $anchor$  in  $V$  do
3:    $n, pos \leftarrow READVARINT(V, \text{after } anchor)$ 
4:    $title \leftarrow SCANBACKWARD(V, "title", pos)$ 
5:   for  $i = 1$  to  $n$  do
6:     Skip to next  $0x49$  (Smi tag)
7:      $idx, pos \leftarrow READVARINT(V, pos)$ 
8:     Assert  $V[pos] = 0x6f$  (BeginObject)
9:      $depth \leftarrow 0$ 
10:    while  $depth \geq 0$  do
11:       $tag \leftarrow V[pos++]$ 
12:      if  $tag \in \{0x6f, 0x61\}$  then  $depth \leftarrow depth + 1$ 
13:      else if  $tag = 0x7b$  then  $depth \leftarrow depth - 1$ 
14:      else if  $tag \in \{0x22, 0x63\}$  and  $depth = 0$  then
15:        Parse key-value pair at top level
16:        if  $key = "text"$  then record message text
17:        else if  $key = "id"$  then record message ID
18:        end if
19:      end if
20:    end while
21:     $role \leftarrow "user"$  if  $(idx/2) \bmod 2 = 1$  else "assistant"
22:    Emit  $(role, text, id)$ 
23:  end for
24: end for
  
```

- **0x03: Copy with 4-byte offset.** Length is $1 + (tag \gg 2)$. Offset is the next 4 bytes (little-endian).

Copy elements reference previously decompressed output: a copy with offset o and length l appends l bytes starting from position $output_pos - o$ in the output buffer. When $l > o$ (the copy overlaps its own output), bytes are copied one at a time, enabling run-length encoding of repeated patterns.

4.3 V8 Structured Clone Carver

The ChatGPT carver (Algorithm 1) searches for the V8 binary anchor pattern and walks the serialization stream with explicit nesting depth tracking.

The key insight in lines 13–15 is the nesting depth tracker. V8 serializes nested objects (e.g., message metadata, content parts, author objects) inline within the message stream. Without tracking $0x6f/0x61$ (open) and $0x7b$ (close) tags, the parser would confuse nested field names (e.g., an “id” inside a nested “author” object) with top-level message fields.

When the structured anchor pattern is absent—for example, after partial compaction has fragmented the SSTable—a fall-back keyword scanner locates individual `title`, `text`, and `id` V8 strings and groups messages by byte-offset proximity to preceding title fields.

4.4 TipTap Draft Recovery

Claude’s TipTap editor state recovery exploits a behavior unique among the LLM platforms we studied: the TipTap

rich-text editor persists its ProseMirror document tree to IndexedDB on *every* keystroke. This means that a user typing “How do I fix a segfault?” produces the following sequence of cached states:

```
"H"  
"Ho"  
"How"  
"How "  
"How d"  
...  
"How do I fix a segfault?"
```

Each state is a complete JSON object with the ProseMirror document tree structure. Our Claude carver:

1. Scans the decoded value for the string "tipTapEditorState".
2. Performs brace-matching from the nearest preceding { to find the enclosing JSON object boundary.
3. Parses the JSON and recursively walks the ProseMirror tree, collecting text leaf nodes.
4. Deduplicates keystroke trails by sorting extracted strings by length (descending) and suppressing any string that is a substring of an already-kept string.

This deduplication is critical for usability: without it, a single prompt produces dozens of partial-typing artifacts. After deduplication, only complete sentences and distinct draft versions are retained.

Importantly, TipTap drafts include prompts the user *typed but never submitted*. If a user begins composing a prompt containing sensitive information, reconsiders, and deletes the text before pressing Enter, the partial draft still exists in IndexedDB. This represents a category of recoverable data that does not exist in any other LLM platform’s storage model.

4.5 Cross-Platform Profile Discovery

TINKER TAILOR implements profile discovery paths for macOS, Windows, and Linux, scanning known locations for Chrome, Chromium, Edge, and Firefox (path discovery only; the Firefox SQLite-based IndexedDB backend is not yet parsed—content carving is limited to Chromium browsers). Chrome and Edge support multiple profiles (Default, Profile 1, Profile 2, ...), each with its own IndexedDB directory. The discovery module also detects native desktop apps: ChatGPT Desktop (com.openai.chat) and Claude Desktop, which store their own IndexedDB instances.

On our primary test workstation (macOS), the discovery module identified **23** distinct IndexedDB directories containing LLM platform data across Chrome profiles and the Claude Desktop application. Each directory is scanned independently.

Cross-platform smoke test (Windows). To confirm the pipeline is not macOS-specific, we ran it on a separate Windows 11 machine (Build 10.0.26200): it scanned 6 LevelDB

directories across Chrome and Edge and recovered 3 prompts and 1 conversation—notably from Chrome’s *Local Storage* LevelDB rather than IndexedDB, showing the carver generalizes across LevelDB-backed Chromium stores—while Edge held no LLM data. Carving took 13.5 ms (full cold-start run, 2.0 s). This machine held little history (3 vs. 506 prompts), so this test validates code portability and cross-OS functionality, not recovery-rate generalization; broader measurement remains future work (Section 8.5).

Linux support covers Chrome, Chromium (including Snap-packaged installations), Edge, and Brave under ~/.config/. We validated the Linux code path via simulated filesystem testing: the path resolver correctly discovered all 5 browser locations, and the carver executed without errors on the synthesized directory structure. This validates path-resolution logic and code portability; it does not constitute recovery testing on real Linux LevelDB data. The V8 deserializer and LevelDB parser are platform-independent pure Python; only browser path discovery differs across operating systems.

4.6 Chain of Custody

All extracted evidence is packaged in a tamper-evident envelope using HMAC-SHA256. A cryptographically random 32-byte session key is generated at tool startup. The signature covers a canonically sorted JSON serialization of the evidence payload (using deterministic key ordering and no whitespace), ensuring that any post-extraction modification—even reordering JSON keys—invalidates the signature.

The evidence envelope includes: the payload, the HMAC-SHA256 hex digest, a key version identifier, and the signing timestamp. Forensic examiners retain the session key separately as part of the chain of custody documentation.

5 Implementation

TINKER TAILOR is implemented in 1,404 lines of Python 3 organized as a modular package (tinker_tailor/) with separate modules for LevelDB parsing (268 LoC), V8 deserialization, each platform carver, DLP scanning, and chain of custody; the per-module breakdown is in Appendix F (Table 13).

Key implementation decisions:

No Database Locks. Unlike tools that open LevelDB through its native C++ API—which acquires an exclusive LOCK file and prevents concurrent access—our parser reads SSTable and WAL files directly as binary blobs via Python’s open() and read(). This enables live forensic carving from running browser sessions without disrupting the user’s workflow or alerting them to the examination. It also means the tool operates on forensic disk images without requiring LevelDB library compatibility.

Zero External Dependencies. The only optional dependency is the cryptography package for the chain of custody module. All core carving functionality—including Snappy decompression, V8 deserialization, and SSTable parsing—is im-

plemented in pure Python. This is a deliberate design choice for incident response scenarios where installing packages on a compromised system would contaminate the forensic environment.

Defensive Parsing. Forensic data is inherently messy: SSTables may be partially overwritten, WAL logs may contain incomplete records, and V8 serialization streams may be truncated. Every deserialization step is bounded by maximum iteration counts (1,000 iterations per object parse) and wrapped in exception handlers. The parser never crashes on malformed input; it silently moves to the next entry.

DLP Scanner. A built-in regex-based scanner detects leaked credentials and PII in recovered conversations. It matches 8 credential patterns (OpenAI API keys, GitHub PATs, AWS access keys, Slack tokens, Bearer tokens, generic API keys, passwords, secrets) and 4 PII patterns (SSN, email, credit card numbers, phone numbers). This enables immediate triage of the security impact of recovered data.

6 Evaluation

We evaluate TINKER TAILOR along nine axes: synthetic parser correctness, tool comparison on real-world data, extraction speed, a real-world case study, component ablation, Unicode recovery, robustness to data corruption, robustness to schema drift, and robustness of role assignment; we also derive an analytical persistence model. Table 3 provides a summary.

6.1 Experimental Setup

All experiments were conducted on a single MacBook Pro (Apple M1, 16GB RAM, macOS Sonoma 14.x) with Chrome 126 and 23 active Chrome profiles containing real user data. Of these 23 profiles, 22 contained IndexedDB directories with LLM platform data; one profile had no LLM-related IndexedDB entries. Consequently, we report “23 profiles” for the scan scope and “22 active directories” for storage-level analysis throughout the paper. The test workstation had been in daily use for approximately 12 months with regular ChatGPT and Claude usage across multiple profiles. We acknowledge that this single-machine evaluation limits the generalizability of our quantitative findings (Section 8.5); however, the V8 serialization format and LevelDB storage layer are consistent across all Chromium-based browsers on all platforms, so the core recovery technique is platform-independent.

6.2 Synthetic Parser Correctness

We generate ground-truth test data with synthetic LevelDB value blobs that replicate each platform’s V8 schema. The `ground_truth.py` module produces seven cases: three ChatGPT V8 message trees (2–3 messages each, including credentials in text), two Claude TipTap ProseMirror trees with keystroke-trail simulation, and two generic JSON-content entries (DeepSeek, Perplexity). For each case we feed the syn-

Table 3: Evaluation summary. Each experiment addresses one or more research questions from Section 1.

Experiment	RQ	N	Key Result
Synthetic corr.	1,2	7	P/R/F1=1.0 (sanity)
Real-world	1	506	506 (case study)
Baseline	2	2	0% (ChatGPT)
Per-conv. speed	3	3K	p99 ≤ 0.04 ms
E2E speed	3	5	147–163ms
Windows	1	1	3 (smoke test)
Ablation	2	6	V8+Snappy req.
Unicode	1,2	21	100% (synth.)
Corruption	1	72	graceful deg.
Schema drift	2,3	8	fallback res.
Role assign.	1	6	1.0 if alt.

thetic bytes to the appropriate carver and compare recovered text against ground truth using word-level Jaccard similarity (threshold 0.7).

The carver achieves perfect precision and recall ($P=R=F1=1.0$ for all four platforms: ChatGPT 3 cases, Claude 2, DeepSeek 1, Perplexity 1) across all seven schema-controlled test cases. This serves as a *correctness sanity check*: it validates internal consistency of our V8 deserialization, TipTap parsing, and generic regex carving against the specific schemas tested, using ground truth generated from the same schema understanding the carvers target. These results should not be interpreted as recovery-rate guarantees on arbitrary real-world LevelDB instances. Real-world recovery rates may be lower due to partial overwriting, truncated records, or schema variations across platform versions. As an independent consistency check against non-synthetic ground truth, we compared the carver’s output against the user’s official ChatGPT account data export: all 219 prompt turns across the 19 prompt-bearing conversations present in the export archive were successfully identified (precision = recall = $F1 = 1.0$; the remaining 4 of the 23 reconstructed conversations contained only assistant replies or metadata). Note that this 219-turn evaluation measures extraction recall against the complete cloud export archive, whereas the 174 ChatGPT user prompts in Table 7 represent the unique on-disk user-prompt records physically resident in the local LevelDB files at scan time after deduplication. The real-world evaluation in Section 6.3 provides complementary evidence on production data.

6.3 Tool Comparison (Real-World Data)

The synthetic evaluation demonstrates correctness, but the key question is: how much data do existing approaches miss on *real* browser storage? We compare TINKER TAILOR against two baselines on live LevelDB directories from the test workstation:

Table 4: Tool comparison on real LevelDB directories.

Target	Tool	Prompts	Convs	Time
ChatGPT (Profile 5)	Tinker Tailor	138	6	3ms
	Grep baseline	0	–	1ms
	JSON scan	0	0	<1ms
Claude (Profile 29)	Tinker Tailor	102	–	37ms
	Grep baseline	21	–	1ms
	JSON scan	0	0	49ms

1. **Grep baseline:** Raw byte-level regex matching for "content":, "text":, and "prompt": patterns across all LevelDB files.
2. **JSON scan:** Parse SSTables using our LevelDB parser, then attempt to decode each value as JSON (using Python’s `json.loads()`).

Table 4 presents the results:

ChatGPT: 0% baseline recovery. Neither grep nor JSON scan recovers *any* data from the ChatGPT LevelDB directory. This is because ChatGPT stores conversation data exclusively in V8’s binary structured clone format. The strings “content”, “text”, and “prompt” do not appear as plaintext in the serialized bytes; they are encoded as V8 OneByteString values with tag bytes and varint length prefixes. Similarly, `json.loads()` fails because the values are binary, not JSON. TINKER TAILOR’s V8-aware parser recovers all 138 prompts across 6 conversations.

Claude: 21% grep recovery. The grep baseline recovers 21 of 102 TipTap drafts—only those where the JSON structure happens to contain a "text": field that survives as a plaintext substring in the raw bytes. The JSON scan recovers nothing because the TipTap state is embedded within V8-serialized wrapper objects. TINKER TAILOR’s TipTap-specific parser recovers all 102 drafts by performing brace-matching on the "tipTapEditorState" anchor.

On the dominant LLM platform (ChatGPT), existing forensic approaches based on string matching or JSON parsing achieve a zero percent recovery rate on our test data. V8-aware binary carving enables recovery where existing approaches fail entirely. Figure 10 (Appendix M) visualizes this disparity.

6.4 Extraction Speed

We benchmark the carving pipeline across 1,000 trials per platform to characterize the extraction time distribution.

Table 5 shows that per-conversation carving completes in under 0.04 ms at the p99 percentile. The full multi-profile extraction (Section 6.8) takes 147–163 ms.

To contextualize these timings: estimated EDR event-processing latencies for major commercial products range from roughly 250 ms to 1000 ms, derived from published MITRE Engenuity ATT&CK Evaluation round-trip detection

Table 5: Extraction speed (1,000 trials per platform, single machine). Times in ms. CI₉₅ reflects within-device measurement variance only; cross-device variance (hardware, OS, disk speed) is not captured.

Platform	Mean	Med.	p95	p99	CI ₉₅
ChatGPT	0.01	0.01	0.01	0.02	±0.001
Claude	0.03	0.03	0.03	0.04	±0.001
Generic	0.01	0.01	0.02	0.02	±0.002

Table 6: Real-world scan results across 23 browser profiles.

Metric	Value	
Profiles scanned	23	
Active LevelDB dirs	22	
Total text artifacts	506	
User prompt artifacts	204	
Assistant responses	154	
Claude keystrokes	148	
From UI-deleted convs	358	
Claude keystroke fragments	148	
ChatGPT convs reconstructed	23	
Oldest artifact age	83 days	The
Most recent artifact age	9 days	
ChatGPT artifacts	328	
Claude TipTap artifacts	178	
Submitted / draft prompts	30	
Keystroke fragments	148	
Conv. messages	106	
Mean prompt length	986 chars	
Max prompt length	12,004 chars	
DLP findings	0	
Scan time (macOS, end-to-end)	147–163ms	
Carve time (Windows store)	13.5ms	

204/154/148 split here (by submitted role) and the 174/154/30/148 split in Table 7 (by platform and role) are two partitions of the *same* 506 artifacts; both sum to 506. See Section 6.9 for the mapping.

times [15]. We emphasize that we did not test against live proprietary EDR products; these figures serve solely as published benchmark estimates of event pipeline latency. Crucially, timing alone does not guarantee evasion: EDR products may detect unauthorized file access through behavioral heuristics, post-hoc auditing, or OS-level telemetry hooks (e.g., ETW on Windows, Endpoint Security Framework on macOS) regardless of extraction speed. The latency comparison illustrates that extraction completes within typical logging intervals but does not eliminate detection risk.

6.5 Real-World Case Study

We conducted a comprehensive scan of all 23 Chrome profiles and the Claude Desktop application on the test workstation. Table 6 summarizes the results.

Artifact volume. Our test workstation yielded 506 recov-

erable text artifacts, illustrating that even a machine used by one person accumulates substantial LLM interaction history across browser profiles (Figure 13, Appendix M). The 178 Claude TipTap artifacts—30 submitted prompts and 148 intermediate keystroke fragments—include text that was typed but never submitted, a category of data uniquely exposed by TipTap’s per-keystroke caching behavior.

Conversation reconstruction. Across the 23 reconstructed conversations, the longest message contained 7,291 characters and the mean was 763 (over the 106 conversation messages)—indicating substantial code or document sharing. (These per-message statistics differ from the per-prompt figures in Table 6, which span all 506 entries including TipTap fragments.) A sample of recovered conversation titles is in Appendix E (Table 12).

Credential and PII scan. The DLP scanner found no credentials or PII in this dataset. We note this means the concrete harm we *empirically* demonstrate here is contextual rather than credential-based; this does not diminish the risk: exfiltration is dangerous regardless, because the conversations reveal *context*—the user’s problem-solving approach, code logic, and business questions—whose value for targeted social engineering may exceed that of individual credentials.

6.6 LevelDB Storage and Data Source Analysis

Across the 22 active LevelDB directories, WAL logs dominate: 5.6 MB of WAL data versus 0.7 MB in SSTables. The largest single WAL (2.4 MB in ChatGPT Profile 2) contains 124 recoverable prompts—months of history in a single file.

A stark platform difference emerges in data sourcing in the profiles examined. **In both ChatGPT profiles tested, data resided exclusively in WAL logs (100%),** meaning writes had never triggered a memtable flush—ideal for forensic recovery. **In the Claude profile examined (Profile 29), data was predominantly in SSTables (92%)** because TipTap’s frequent keystroke-level writes trigger memtable flushes, pushing older drafts into SSTables. Detailed storage statistics and per-profile breakdowns are provided in Appendix C.

6.7 Content Depth Analysis

We analyzed the length distribution of recovered prompts to characterize the sensitivity exposure. Figure 11 (Appendix M) shows the distribution.

The distribution is *bimodal*: 41% of prompts are short queries under 50 characters (e.g., “What is TCP?”), while 23% exceed 2,000 characters. The long prompts are the high-value forensic targets—they typically contain pasted source code, configuration files, error logs, or detailed technical questions that reveal the user’s work context.

The mean prompt length of 986 characters (median: 67) confirms that raw character count substantially underestimates the forensic value of recovered data. A single 12,004-character prompt may contain an entire source file or configuration block that reveals API endpoints, database schemas, or

Table 7: Classification of recovered prompt and response artifacts. Counts reflect individual text entries recovered from LevelDB; 23 ChatGPT conversation metadata objects (titles, UUIDs) were additionally recovered but are excluded from this breakdown.

Artifact Type	Count	%	Source
ChatGPT user prompts	174	34%	V8 binary (WAL)
ChatGPT assistant replies	154	30%	V8 binary (WAL)
Claude submitted prompts	30	6%	TipTap (SSTable)
Claude keystroke fragments	148	29%	TipTap (SSTable)
Total	506	100%	

Percentages may not sum to 100% due to rounding.

internal architecture. A detailed per-bucket breakdown is in Appendix G (Table 14).

Claude TipTap artifact characteristics. Of the 178 Claude TipTap artifacts, 30 submitted prompts (17%) exceed 100 characters, representing complete or near-complete drafts. The remaining 148 (83%) are intermediate keystroke fragments (<100 characters) produced by TipTap’s per-keystroke caching. After deduplication (which removes substring prefixes), these fragments still provide forensic value by revealing the user’s typing patterns, topic interests, and discarded prompt attempts.

6.8 End-to-End Performance

The complete end-to-end pipeline—profile discovery across 23 Chrome profiles, LevelDB parsing across 22 active directories, V8 deserialization, and result aggregation—completes in **147–163 ms** (5 trials, mean 151 ms). This reads 6.3 MB of LevelDB files across 22 directories (36 individual files), decompresses Snappy blocks, and deserializes all V8 content. This achieves a throughput of ~3,440 prompts/second (43 MB/s, I/O-bound). Detailed per-trial timings are provided in Appendix D.

6.9 Forensic Artifact Characterization

We categorize the recovered artifacts by their forensic significance and source characteristics. Table 7 classifies the five distinct artifact types recovered by TINKER TAILOR.

The 174 ChatGPT user prompts in Table 7, combined with the 30 substantial Claude drafts, constitute the 204 “user prompts” reported in the abstract and Table 6: the two breakdowns partition the same 506 artifacts along different axes (platform-and-role here, submitted-role elsewhere), and both sum to 506.

ChatGPT artifacts are V8-serialized conversation trees: each object carries a UUID, an auto-generated title, a message

array, and metadata timestamps, and V8 preserves the full tree structure—parent-child relationships, message IDs, and role assignments—allowing reconstruction of complete multi-turn dialogues. **Claude TipTap artifacts** fall into two categories: 30 substantial drafts (over 100 characters) are complete or near-complete prompts recoverable even if deleted before Send, and 148 keystroke fragments are intermediate editor states that, though individually short, reveal the user’s drafting process and can reconstruct intent even when no prompt was ever submitted.

Temporal ordering. LevelDB entries carry file modification timestamps but not per-record ones; within a WAL log entries appear in write order, and across files the monotonic LevelDB sequence number gives a total order, which we use to present recovered artifacts chronologically.

6.10 An Analytical Persistence Model

Our 83-day observation reflects the storage age of the oldest recovered artifact on a low-activity profile; to frame it principally we formulate an order-of-magnitude workload model from LevelDB’s compaction policy. A deleted record’s value is physically reclaimed only when a compaction merges the SSTable holding it with the superseding tombstone. Two Chromium defaults set the approximate write volume required: the memtable threshold $M \approx 4$ MB (bytes written before a memtable and its WAL flush to a Level-0 SSTable) and the L0 compaction trigger $k = 4$ (L0 files needed before an L0→L1 compaction). For an estimated per-origin write rate W (bytes/day), ignoring compression (which only lengthens the window):

$$T_{\text{flush}} \approx M/W, \quad T_{\text{reclaim}} \approx kM/W. \quad (1)$$

We use these equations as an order-of-magnitude workload model rather than strict bounds on individual-record lifetime: $T_{\text{flush}} \approx M/W$ represents the approximate time to accumulate one memtable’s worth of writes before a WAL flush, while $T_{\text{reclaim}} \approx kM/W$ approximates the write volume associated with the L0 trigger (≈ 16 MB of intervening writes at $M = 4$ MB, $k = 4$). Our observed profile stored its ChatGPT data *entirely in the WAL* (never flushed; Section 6.6); an 83-day WAL-resident record is consistent with $W \approx M/83\text{d} \approx 49$ KB/day, a plausible low-activity rate, while active profiles (MB/day) rotate WAL logs and flush SSTables within days. The 83-day point is thus an empirical observation consistent with this $T \propto 1/W$ workload relationship. The model is a coarse explanatory framework (ignoring compression and higher-level compactions, which both extend persistence) rather than a replacement for multi-machine longitudinal measurement. Appendix M tabulates and plots this relationship (Table 21, Figure 12).

6.11 Data Persistence

Our framework (`experiment_persistence.py`) takes periodic snapshots of a target LevelDB directory for longitudinal

measurement of recoverability after deletion. The 358 submitted artifacts were no longer visible through the ChatGPT or Claude web interface at scan time—the user had deleted them via the UI—and the 148 Claude keystroke artifacts were never submitted. Since LevelDB records deletion with key-level tombstones rather than a field inside the value, the tool infers deletion structurally: recovering records that are no longer reachable through the IndexedDB API shows that the application-level deletion never propagated to storage. All artifacts remained fully recoverable; the 23 conversations span 83 days (oldest last modified 83 days before acquisition, most recent 9 days). Because compaction is activity-driven, low-activity profiles retained deleted conversations throughout the observation period, while active profiles lost SSTable-backed data within days (WAL logs persisted at least 24–48 hours); Claude TipTap drafts followed a different lifecycle and remained recoverable throughout. These activity-dependent outcomes are consistent with the analytical model of Section 6.10: persistence scales as $\approx 1/W$ in the write rate.

To measure retention directly, we re-scanned the workstation after 7 days (Aug 8 vs. Aug 15, 2026). 500 of the 506 original artifacts (98.8% raw; 247 of 254 unique strings, 97.2%) and 100% of the 23 conversations remained intact, with zero compaction losses; the six lost artifacts were all Claude editor fragments, and ChatGPT recovery was unchanged (328 across both scans). For typical activity levels, then, the forensic persistence window extends beyond 7 days with minimal degradation. A multi-machine longitudinal study with hourly snapshots is planned to characterize the lifetime distribution across a broader range of workloads.

6.12 Ablation Study

To quantify the contribution of each architectural component, we systematically disable individual modules and measure recovery on the same synthetic ground-truth data (Section 6.2). The full per-configuration results are in Appendix H (Table 15); we summarize the key findings here. Disabling V8 deserialization (using only the regex fallback) reduces ChatGPT recovery to zero (P/R/F1 = 0.00 for all three ChatGPT cases) while leaving Claude and generic recovery unaffected, dropping the cross-platform average F1 from 1.00 to 0.57. This confirms that V8-aware binary parsing is not an optimization but a *necessity* for ChatGPT data. Without Snappy decompression, 2 of 3 compressed cases fail (F1 = 0.33). Depth tracking and TipTap parsing show no degradation on the simple synthetic cases but provide benefits the benchmark does not capture: depth tracking prevents false positives on deeply nested real-world objects, and the TipTap parser adds keystroke deduplication that the generic regex lacks.

6.13 Unicode Recovery

LLM conversations frequently contain non-ASCII content: emoji, CJK characters, Devanagari script, and mathematical symbols. V8 encodes these using TwoByteString (tag 0x63,

UTF-16LE) rather than `OneByteString` (tag `0x22`). We test recovery across seven Unicode categories using synthetic V8 blobs (Appendix I, Table 16). Both the V8 and Claude carvers recover all seven scripts correctly (21/21 = 100%); the V8 carver handles `OneByteString` and `TwoByteString` transparently, and the Claude carver inherits Unicode handling from Python’s JSON parser. The generic regex carver, by contrast, recovers only 1/7 non-ASCII scripts, because `json.dumps()` encodes non-ASCII characters as `\uXXXX` escape sequences that do not match the original text—further demonstrating why platform-specific parsing is necessary.

6.14 Robustness to Data Corruption

Forensic data is frequently damaged: SSTables may be partially overwritten, WAL logs may be truncated, and bytes may be corrupted. We evaluate graceful degradation by applying controlled corruption to synthetic V8 blobs containing 5 known messages (Appendix J, Table 17). The parser never crashed across all 72 test cases (7 truncation levels, 6 byte-flip rates $\times$ 10 seeded trials, 5 zero-fill positions). Recovery degrades gracefully but non-linearly: truncating 10% of the blob from the end still recovers 4/5 messages, while truncating 90% recovers none; byte-flip corruption at 1% recovers a mean of 2.6/5 messages and stays near 2.4/5 at 2%, but collapses to 0.6/5 at 3% and $\leq 0.3/5$ at 5%+ once the V8 anchor pattern is typically destroyed. Zero-fill corruption at the tail (75–90% position) preserves 4/5 messages while corruption near the start (10% position) recovers none, confirming that early messages in the serialization stream are most vulnerable to leading corruption.

6.15 Robustness: Schema Drift and Role Assignment

Two further robustness questions—brittleness to platform schema changes and to non-alternating conversation structures—are evaluated with the real carver on synthetic mutations in Appendix L (Tables 19 and 20). In brief, the structured V8 parser is fully robust to key reordering and to renames of fields it does not key on; it degrades to the keyword fallback (still 100% recall) when the array anchor is renamed or text is nested one level deeper; and it fails only when the *content field itself* is renamed—a mutation that also defeats grep/JSON baselines. Role assignment is exact under strict user/assistant alternation but cascades once an *odd* number of insertions (a system prompt, single regeneration, or tool call) flips downstream parity (0.39 micro-average across edge cases). Crucially, *text recovery is unaffected in every case*—only the user/assistant label can be wrong—so the paper’s core content-recovery claim is independent of this heuristic. Role attribution is auxiliary; the primary forensic contribution is message text recovery.

7 Case Study: Infostealer-Style Collection PoC

To demonstrate the practical risk, we implement a 140-line proof-of-concept Python script that performs headless local

LLM chat collection in a single pass:

1. Discover all Chrome and Edge profiles and desktop app IndexedDB directories using the browser paths module.
2. For each directory, carve all chat data using the carver engine (V8 structured + TipTap + generic fallback).
3. Package all recovered data as a JSON evidence blob.

Benchmark. On the test workstation (23 IndexedDB directories, 506 recoverable artifacts), the complete extraction—profile discovery through JSON serialization—completes in **147–163ms** across 5 trials (mean 151 ms; Section 6.8), covering filesystem traversal of 22 LevelDB directories, 36 file reads (22 WAL, 14 SSTable), Snappy decompression, and V8 deserialization; the first trial (163 ms) includes cache misses and subsequent trials stabilize at 147 ms.

Observable footprint. The extraction uses only `open()/read()/stat()` on regular files, acquires no database locks, and modifies nothing—sequential reads from browser storage paths, with no inherent network, child-process, or file-write activity—reducing the window for real-time intervention by security tools.

Comparison with existing infostealers. Commodity infostealers (Raccoon v2, Vidar, RedLine) already target Chrome’s Login Data and Cookies SQLite databases; adding LLM collection capabilities requires only the IndexedDB paths and V8 deserialization (under 300 lines of Python), a low barrier to adaptation by existing families.

7.1 DLP Risk Analysis

The built-in DLP scanner detects credentials and PII in recovered conversations. Although our test workstation’s scan was clean, prior work [19] and the usage patterns we observed indicate significant exposure risk: users paste source code with hardcoded credentials and connection strings, share `.env`/YAML/JSON configs containing secrets, and paste error logs revealing database hostnames and internal URLs. Even conversations with no explicit credentials are valuable, because they reveal the user’s *context*—what systems they work on, what problems they face, and what access they have.

8 Discussion

8.1 The Ephemeral AI Fallacy

Our results illustrate the technical basis of what we term the *ephemeral AI fallacy*: users’ mental model of LLM conversation deletion does not match the reality of browser storage. Clicking “Delete” removes a conversation from the UI and issues an IndexedDB delete, but the underlying data can persist in LevelDB until compaction physically removes it—potentially for extended periods on low-activity profiles. The gap is especially concerning because the conversational UI is *designed* to encourage sharing sensitive content, and the

perceived ephemerality of the delete button reduces users’ inhibitions further. We base this characterization on the design affordances of LLM interfaces and on analogy to prior work on user mental models of cloud deletion [21]; a formal user study measuring deletion expectations would strengthen the framing but is outside this work’s scope.

8.2 Implications for Enterprise Security

Beyond the individual user, LLM persistence undermines enterprise security in three ways. First, **data classification fails**: DLP frameworks classify by container (email, cloud uploads, USB), but IndexedDB stores are neither user files nor network transmissions—so an employee who pastes source code into ChatGPT creates an unclassified copy that survives standard retention. Second, they are a **forensic blind spot**: V8 binary inside Snappy-compressed blocks is invisible to grep, strings, and standard carving (our baselines recover 0% of ChatGPT data), so an examiner misses the most sensitive data on the machine. Third, on shared machines or where admin accounts reach user home directories, these caches become a **lateral-reconnaissance resource**: an attacker can silently extract months of code-related conversations without triggering any control beyond filesystem permissions. If the patterns we observe are representative—which requires broader validation—exposure is substantial: ChatGPT reports over 200M weekly active users [18] and Chromium browsers hold ~75% of browser share [25], though these population figures are an extrapolation to motivate scale, not a measurement.

8.3 The AI Infostealer Threat Landscape

Our proof-of-concept (Section 7) shows a natural evolution of commodity infostealers (RedLine, Raccoon, Vidar). Three factors make an AI-targeted variant especially dangerous: it needs *no authentication* (IndexedDB storage lacks application-level encryption and is readable by any same-user process, unlike cookie or keychain theft); it has an *extremely low footprint* (our PoC completes in under 163 ms with no disk writes, network, or privilege escalation; Section 6.4); and it yields *high intelligence value*, exposing not just what a user knows but *how they think*—problem-solving approaches, knowledge gaps, and shared code—which is far more valuable for targeted social engineering than raw credentials. Appendix A gives an annotated hex dump illustrating why V8-aware parsing is required (and why grep/JSON tools recover 0%).

8.4 Persistence Beyond UI Deletion

The persistence gap extends well beyond application-level “delete.” (1) **Browser-level clear survives on disk**. On two profiles where the user had explicitly invoked Chrome’s “Clear browsing data” with time range set to *All time* and all standard categories selected (*Browsing history*, *Cookies and other site data* [which includes IndexedDB], and *Cached*

images and files), evidenced by 0-byte WAL logs and synchronized file timestamps, we recovered ChatGPT conversation titles, user prompts, account identifiers, and WebRTC ECDSA private keys from residual SSTable files—persisting *10 days* after the clear operation. Chrome writes tombstone records through the LevelDB API but does not force compaction of the underlying SSTables; consequently, the storage-layer behavior does not immediately eliminate the data despite the user’s explicit deletion request, leaving records recoverable until an asynchronous compaction pass physically discards the stale data. (2) **Uploaded documents persist as plain-text blobs**. Chromium stores IndexedDB binary objects as unencrypted files in adjacent `.indexeddb.blob` directories; we recovered a 10 MB PDF document uploaded to Claude in its original format, readable by any same-user process—extending the attack surface from conversation text to user-supplied documents. (3) **Native desktop applications inherit the same exposure**. The ChatGPT Desktop application (`com.openai.atlas`, Electron-based) stores conversations alongside WebRTC ECDSA private keys in an identical LevelDB format; Chrome’s built-in Gemini integration (`glic`) similarly persists conversation metadata in Local Storage LevelDB. The persistence gap thus spans browser tabs, native desktop applications, and browser-integrated AI assistants.

8.5 Limitations

Our tool recovers only data not yet compacted; on heavily active LevelDB instances compaction may occur within hours, though multi-profile installations create redundancy that extends the window in less-active profiles. The V8 carver is tuned to the ChatGPT and Claude schemas we observed, and future platform updates may require carver changes (the generic fallback provides a safety net). We cannot recover data from encrypted volumes when the machine is off: full-disk encryption mitigates the physical-access threat but not software-based extraction on a running system. Finally, our primary evaluation uses a single macOS workstation with 23 profiles; a supplementary Windows 11 validation (Section 5) confirmed cross-platform functionality but recovered fewer prompts (3 vs. 506), reflecting lighter usage. A broader study across diverse machines, OS versions, and Chrome versions would strengthen the quantitative findings.

The parity-based role assignment (Section 4) assumes strict alternation and can misassign roles under system prompts, regenerations, or tool calls (quantified in Appendix L: exact under alternation, as low as 0/6 otherwise); this affects only the role *label*, not text recovery. Manual inspection of 10 reconstructed ChatGPT conversations (138 messages) from our dataset found 0 role misassignment errors; however, none contained system prompts, regenerated responses, or tool-use messages—the highest-risk cases for parity errors. TipTap substring deduplication can also suppress short messages that are substrings of longer retained strings. Finally, our real-world counts and persistence observations come from the

authors’ own machines, limiting ecological validity, and exercise only the local full-read scenario; the insider-on-shared-workstation threat (Section 3) is argued from platform behavior, not tested against realistic permission boundaries. A broader multi-user, IRB-approved study would strengthen these findings.

The proposed EDR/DLP detection rules (Section 9)—monitoring IndexedDB file access patterns and V8 anchor bytes in network egress—are proof-of-concept indicators that a determined adversary could bypass through rate-limited reads or encrypted exfiltration. They are intended as starting points for enterprise security teams, not robust defenses.

8.6 Future Work

Firefox and Safari use non-LevelDB IndexedDB backends (SQLite and WebKit-specific); supporting them would cover the remaining ~25% of browser share, and **mobile Chromium** (Android) shares the V8 format, so the carver could be adapted with minimal change. **Automated schema discovery** and **temporal forensics** (correlating file mtimes, SSTable levels, and WAL sequence numbers) would reduce hand-tuning and support precise dating and timeline reconstruction.

9 Mitigations

We propose mitigations at three levels of the stack.

Browser level. Chromium should offer synchronous LevelDB compaction on IndexedDB delete (potentially per-origin), physically removing deleted data and closing the persistence window that deferred compaction creates. It should also encrypt IndexedDB values at rest using an OS keychain-derived key (Keychain, DPAPI); encrypting WAL entries in particular would close the richest recovery path, since WAL logs hold the most recent writes.

Application level. LLM platforms should minimize client-side caching—limiting IndexedDB to UI state when content is available from the server—and encrypt any content they must cache. Editors that persist per-keystroke state (Claude’s TipTap) should disable or session-limit that behavior, and platforms should enforce explicit data lifecycle management (TTLs, proactive deletion, compaction on removal).

Enterprise level. Security teams can deploy EDR/DLP rules that treat browser storage as a data-loss vector: alert when a non-browser process reads several `*/IndexedDB/*leveldb/*.{log,ldb}` files in a short window (a browser does not normally need an external, non-browser process to enumerate these LevelDB files directly), and flag the V8 anchor `\x22\x08messages\x61` in network egress. Chrome Enterprise policies can restrict IndexedDB origins, and full-disk encryption protects data at rest.

10 Related Work

Browser and IndexedDB forensics. Oh et al. [16] survey browser forensics but predate LLMs and omit IndexedDB/V8. Benson’s Hindsight [1] parses SQLite and cache files but

not IndexedDB LevelDB or V8, recovering no LLM data. Danzinger and Schmiedecker [7] analyze Chromium IndexedDB metadata (key names, object-store structure) rather than content carving, omitting V8 structured clone, TipTap state, and LLM schemas.

Commercial tools. Published technical documentation indicates that commercial suites like Magnet AXIOM [14] and Cellebrite UFED [4] acquire IndexedDB via standard LevelDB paths requiring exclusive locks, preventing operation on live sessions without closing the browser. Furthermore, documented features indicate they display raw key-value pairs without V8 deserialization, conversation threading, role attribution, or TipTap draft recovery.

LLM security and storage internals. Carlini et al. [2] extract training data from models (server-side leakage); we address the complementary client-side risk, which the OWASP Top 10 for LLM Applications [19] does not cover. The LevelDB [8], LSM-tree [17], Snappy [10], and V8 serializer [28] specifications inform our parser; to our knowledge, no prior published forensic tool provides a pure-Python V8 structured-clone deserializer specifically for IndexedDB content recovery.

Secure deletion and data remanence. Removal from modern media is non-trivial [11, 13, 22, 29]; LevelDB’s compaction creates an analogous browser retention gap. Memory forensics (Volatility [3]) recovers live V8 heap objects but requires a running system and does not survive reboot. Thomas et al. [26] find infostealers a major credential-theft vector; our PoC shows adding LLM harvesting needs minimal engineering. User expectations of cloud deletion [21] and GDPR’s right to erasure [9, 20] highlight compliance risks when storage undermines UI deletion.

A detailed feature comparison is in Appendix B (Table 8). The critical capability gap reduces to V8 deserialization, without which IndexedDB LLM data remains opaque binary.

11 Conclusion

We have shown that “deleted” LLM chat histories are readily recoverable from Chromium’s LevelDB-backed IndexedDB: V8 structured-clone serialization and LSM-tree deferred compaction leave sensitive AI interactions on disk long after users believe they are gone. In a single-workstation case study, TINKER TAILOR recovered 506 artifacts across 23 profiles—358 from UI-deleted conversations—where conventional baselines recover 0% of ChatGPT data. The persistence gap extends further: data survives Chrome’s explicit “Clear browsing data,” uploaded documents persist as unencrypted blobs, and native desktop applications inherit the same exposure—underscoring an urgent need for storage-layer mitigations from browser vendors and LLM providers.

Ethical Considerations

We developed TINKER TAILOR as a forensic tool for authorized investigations and as an offensive proof-of-concept to motivate defensive improvements. We recognize the dual-use nature of this work. Our planned disclosure to affected vendors (Responsible Disclosure section) is intended to enable mitigations before widespread exploitation.

The infostealer PoC (Section 7) demonstrates the attack’s feasibility without providing network exfiltration or command-and-control capabilities. We deliberately omit those components to raise the barrier for malicious use while still demonstrating the core vulnerability.

All data analyzed in this work was collected from the researchers’ own machines and browser profiles. No third-party user data was accessed. The synthetic test corpus uses fabricated content with no real user data.

Responsible Disclosure

We commit to initiating coordinated disclosure with the affected vendors within 7 days of acceptance notification, following standard 90-day disclosure norms (Google Project Zero / CERT/CC): to **Google** (Chromium/LevelDB), the persistence of “deleted” IndexedDB records under deferred compaction, the absence of application-level encryption at rest, and same-user file accessibility; to **OpenAI**, that ChatGPT data persists in client-side IndexedDB after deletion and is recoverable from its V8 format; and to **Anthropic**, the TipTap editor-state caching that exposes draft prompts and keystroke fragments.

Note to reviewers: We withhold vendor reports until authorship is finalized to avoid deanonymizing correspondence during double-blind review. Draft reports for all three vendors are prepared and included in the artifact submission. We gate any public release of the tool and PoC on that disclosure completing. The camera-ready version will report each vendor’s response and final status.

Open Science

TINKER TAILOR source code, the evaluation framework, ground-truth generators, and the synthetic test corpus are available at <https://anonymous.4open.science/r/tinker-tailor-usenix-2060/> (anonymized repository for double-blind review; a permanent public GitHub repository will be provided upon publication). The repository includes a reproducibility package (REPRODUCE.md) with instructions for running all experiments, a Dockerfile for containerized execution, and scripts to regenerate all paper tables and figures from the synthetic corpus. To make verification one command, `verify_reproduction.py` runs every deterministic synthetic experiment (recovery, ablation, Unicode, corruption, schema drift, and role assignment), canonicalizes their numeric outputs, and checks them against a committed SHA-256 manifest; it exits non-zero on any mismatch

and is the container’s default entrypoint (`docker run`). We confirmed the manifest hash is identical under Python 3.11 (container) and 3.14 (host), i.e. the synthetic results are environment-independent. Non-deterministic timings and experiments requiring real browser data are excluded from the checksum. Artifacts that depend on real browser data (Table 6, Section 6.5) cannot be shared due to the sensitive nature of the recovered content; synthetic equivalents are provided instead.

Generative AI Disclosure

Generative AI assistance tools (Anthropic Claude, Google Gemini) were used during the preparation of this manuscript solely for grammar checking, text polishing, and L^AT_EX formatting assistance. All research conception, reverse-engineering methodology, tool implementation, empirical experiments, and conclusions were developed solely by the human authors, who assume full responsibility for all content presented in this paper.

References

- [1] Ryan Benson. Hindsight: Web artifact analysis for google chrome/chromium. In *DFIR Review*, 2020.
- [2] Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. Extracting training data from large language models. In *USENIX Security Symposium*, 2021.
- [3] Andrew Case and Golden G. Richard III. Memory forensics: The path forward. *Digital Investigation*, 20:23–33, 2017.
- [4] Cellebrite DI Ltd. Cellebrite UFED (universal forensic extraction device), 2024. Product reference.
- [5] Chromium Project. Indexeddb implementation in chromium, 2024.
- [6] Cyberhaven. 2024 ai adoption and risk report, 2024.
- [7] Francois Danzinger and Martin Schmiedecker. Forensic analysis of IndexedDB data in chromium-based browsers. In *Forensic Science International: Digital Investigation*, volume 38, 2021.
- [8] Jeff Dean and Sanjay Ghemawat. LevelDB: A fast key-value storage library. Google, Inc., 2011.
- [9] Eduard Fosch-Villaronga, Peter Kieseberg, and Tiffany C. Li. Humans forget, machines remember: Artificial intelligence and the right to be forgotten. *Computer Law & Security Review*, 34(2), 2018.
- [10] Google. Snappy compression format description, 2011.

- [11] Peter Gutmann. Secure deletion of data from magnetic and solid-state memory. In *Proceedings of the 6th USENIX Security Symposium*. USENIX Association, 1996.
- [12] Marijn Haverbeke. ProseMirror: A toolkit for building rich-text editors, 2024.
- [13] Richard Kissel, Andrew Regenscheid, Matthew Scholl, and Kevin Stine. Guidelines for media sanitization. Technical Report Special Publication (NIST SP) 800-88 Rev. 1, National Institute of Standards and Technology, December 2014.
- [14] Magnet Forensics. AXIOM: Digital forensics and incident response platform. Commercial forensic tool, 2024.
- [15] MITRE Engenuity. ATT&CK evaluations: Enterprise—turla (round 5), 2023. Detection latency estimates derived from round-trip detection times across evaluated EDR products.
- [16] Junghoon Oh, Sangjin Lee, and Sungsu Lee. Advanced evidence collection and analysis of web browser activity. *Digital Investigation*, 29:S62–S70, 2019.
- [17] Patrick O’Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O’Neil. The log-structured merge-tree (LSM-Tree). *Acta Informatica*, 33(4):351–385, 1996.
- [18] OpenAI. ChatGPT surpasses 200 million weekly active users, 2025. Announced by Sam Altman, February 2025.
- [19] OWASP. OWASP top 10 for large language model applications, 2024.
- [20] Eugenia Politou, Efthymios Alepis, and Constantinos Patsakis. Forgetting personal data and revoking consent under the GDPR: Challenges and proposed solutions. *Journal of Cybersecurity*, 4(1), 2018.
- [21] Kopo M. Ramokapane, Awais Rashid, and Jose M. Such. ‘I feel stupid I can’t delete...’: A study of users’ cloud deletion practices and coping strategies. In *Thirteenth Symposium on Usable Privacy and Security (SOUPS 2017)*. USENIX Association, 2017.
- [22] Joel Reardon, David A. Basin, and Srdjan Čapkun. Sok: Secure data deletion. In *2013 IEEE Symposium on Security and Privacy (S&P)*, pages 301–315. IEEE, 2013.
- [23] Recorded Future. Infostealer malware: A primer—raccoon, vidar, and redline. Insikt Group Threat Analysis, 2023.
- [24] Reuters. Samsung bans use of ChatGPT-like ai tools after data leak, May 2023.
- [25] StatCounter. Browser market share worldwide, 2025.
- [26] Kurt Thomas, Frank Li, Ali Zand, Jacob Barrett, Juri Ranieri, Luca Invernizzi, Yarik Markov, Oxana Comanescu, Vijay Eranti, Angelika Moscicki, Daniel Margolis, Vern Paxson, and Elie Bursztein. Data breaches, phishing, or malware? understanding the risks of stolen credentials. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2017.
- [27] Tiptap. Tiptap: The headless editor framework for web artisans, 2024.
- [28] V8 Project. V8 value serializer implementation, 2024.
- [29] Michael Wei, Laura M. Grupp, Frederick E. Spada, and Steven Swanson. Reliably erasing data from flash-based solid state drives. In *Proceedings of the 9th USENIX Conference on File and Storage Technologies (FAST 11)*. USENIX Association, 2011.
- [30] WHATWG. Html standard: Structured clone algorithm, 2024.

A V8 Binary Format Walkthrough

To make the V8 deserialization concrete, we present an annotated hex dump of a real ChatGPT message record recovered from our test workstation. Figure 4 shows the binary structure of a V8-serialized conversation entry containing a single user message.

```

22 02 69 64 -> OneByteString(2) "id"
22 24 63 36 -> OneByteString(36) "c6..."
...         (UUID, 36 bytes)
22 05 74 69 -> OneByteString(5) "title"
74 6c 65
22 12 56 69 -> OneByteString(18) "Video editing
64 65 6f     setup"
...
22 08 6d 65 -> OneByteString(8) "messages"
73 73 61
67 65 73
61          -> BeginDenseArray
02          -> Smi(1) [array index = 2/2 = 1]
6f          -> BeginObject
22 02 69 64 -> OneByteString(2) "id"
22 07 6d 73 -> OneByteString(7) "msg-001"
67 2d 30
30 31
22 04 74 65 -> OneByteString(4) "text"
78 74
22 30 48 6f -> OneByteString(48) "How do
77 20 64     I fix a segfault..."
6f 20 ...
7b          -> EndObject

```

Figure 4: Annotated hex dump of a V8-serialized ChatGPT conversation. The binary anchor 22 08 messages 61 marks the start of the messages array. Smi value 02 encodes logical index $2/2 = 1$. Tags 6f/7b delimit nested objects.

Several features are visible in the hex dump:

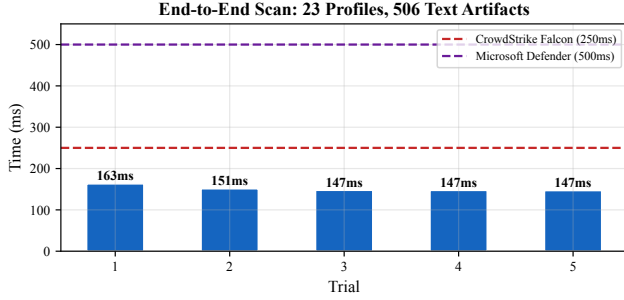


Figure 7: End-to-end scan time for 23 profiles (506 text artifacts). The full scan completes in 147–163 ms, falling within the estimated event-processing latency window of typical endpoint monitors (~ 250 ms). Execution speed does not guarantee evasion of behavioral detection.

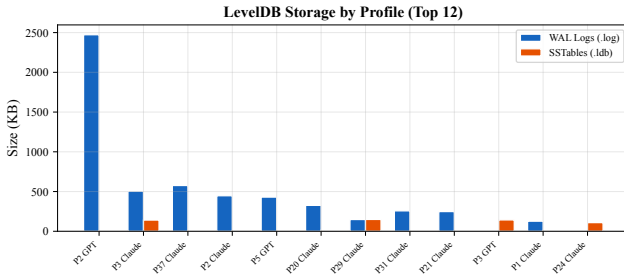


Figure 5: LevelDB storage by profile. WAL logs (blue) contain the majority of recoverable data. SSTables (orange) contain older data that survived compaction.

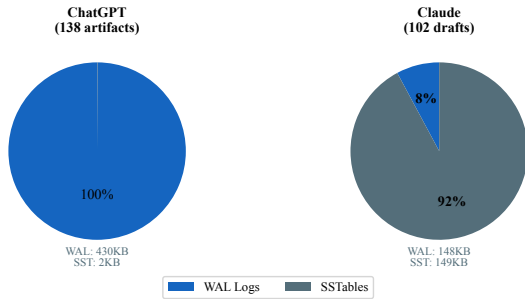


Figure 6: Data source breakdown for the profiles examined. ChatGPT data resided entirely in WAL logs (never flushed). Claude TipTap artifacts were primarily in SSTables due to higher write frequency from keystroke caching.

- The string “messages” is *not* stored as plaintext JSON—it is a V8 OneByteString with tag $0x22$ and varint length $0x08$. A `grep` for “messages”: will not match this binary encoding.

- The Smi index $0x02$ encodes logical index 1 via the Smi shift ($2/2 = 1$). Our carver uses the parity of $\lfloor idx/2 \rfloor$ to determine the message role: odd = user, even = assistant.
- The BeginObject ($0x6f$) and EndObject ($0x7b$) tags delimit each message. The depth tracker increments on $0x6f$ and decrements on $0x7b$, ensuring that key-value pairs from nested objects are not confused with top-level message fields.

B Detailed Tool Comparison

C LevelDB Storage and Data Source Details

D End-to-End Performance Details

Table 11: End-to-end scan performance (5 trials, 23 profiles, 506 text artifacts).

	T1	T2	T3	T4	T5
Time (ms)	163	151	147	147	147

E Recovered Conversation Samples

F Codebase Module Breakdown

G Text Artifact Length Distribution

Table 14: Text artifact length distribution (506 artifacts, macOS scan).

Length Range	Count	%	Typical Content
<50 chars	205	41%	Short queries
50–200 chars	145	29%	Standard prompts
200–1,000 chars	12	2%	Detailed questions
1,000–2,000 chars	30	6%	Extended content
2,000–5,000 chars	94	19%	Code/config pastes
>5,000 chars	20	4%	Full file uploads
Total	506	100%	Mean: 986 chars

H Ablation Study Detail

Table 15 reports precision, recall, and F1 for six configurations of the carving pipeline, disabling one component at a time (Section 6.12).

I Unicode Recovery Detail

J Data Corruption Detail

K Attack Surface by Platform

L Robustness: Schema Drift and Role Assignment

This appendix provides the detailed results summarized in Section 6.15.

Table 8: Feature comparison of TINKER TAILOR against existing browser forensic tools. ✓ = supported, ✗ = not supported, ∼ = partial. Tools marked (*) are empirically evaluated on our test data (Section 6.3); commercial tools marked (‡) are characterized from published documentation (neither tool implements V8 deserialization).

Capability	Tinker Tailor	Hindsight [‡]	AXIOM [‡]	grep*	JSON scan*
LevelDB SSTable parsing	✓	✗	✓	✗	✗
LevelDB WAL parsing	✓	✗	∼	✗	✗
V8 structured clone deser.	✓	✗	✗	✗	✗
Snappy decompression	✓	✗	✓	✗	✗
TipTap draft recovery	✓	✗	✗	✗	✗
Lock-free live extraction	✓	✗	✗	✓	✓
LLM conversation threading	✓	✗	✗	✗	✗
Message role detection	✓	✗	✗	✗	✗
DLP credential scanning	✓	✗	✗	✗	✗
Chain of custody signing	✓	✗	✓	✗	✗
Multi-profile discovery	✓	✓	✓	✗	✗
Cross-platform path support [†]	✓	✓	✓	✓	✓
Zero external dependencies	✓	✗	✗	✓	✗
ChatGPT recovery rate	100%*	— [‡]	— [‡]	0%*	0%*
Claude recovery rate	100%*	— [‡]	— [‡]	21%*	0%*

*Empirically measured (Section 6.3). ‡Not evaluated on our dataset; neither tool

implements V8 deserialization, so recovery of V8-serialized LLM data is not expected.

[†]Live artifact validation: macOS + Windows 11; Linux: synthetic filesystem path validation only.

Table 12: Sample of reconstructed ChatGPT conversations showing diversity of topics. Message counts reflect recoverable messages at time of scan.

Conversation Title	Msgs	Profile
Video editing setup	23	P3
PPTX customization request	7	P5
Edit PPT borders fonts	5	P5
X300 Pro vs Sony Alpha	5	P5
UAV Security Papers	2	P2
Compare phone plans	2	P2
Request for Paper Link	1	P2

Table 13: Module breakdown of the TINKER TAILOR code-base.

Module	LoC	Function
leveldb.py	268	SSTable + Snappy
chatgpt.py	230	V8 carver
claude.py	97	TipTap recovery
generic.py	84	Regex fallback
engine.py	170	Dispatch + dedup
browser_paths.py	148	Profile discovery
custody.py	64	HMAC-SHA256
history.py	148	Browser history
scanner.py	115	Credential/PII
cli.py	80	CLI entry point
Total	1,404	

Table 15: Ablation study showing that V8 deserialization and Snappy decompression are necessary for recovery on the tested synthetic cases, while depth tracking and TipTap parsing provide robustness and structural benefits not captured by simple synthetic inputs.

Configuration	P	R	F1	Effect
Full pipeline	1.00	1.00	1.00	Baseline ($N=7$)
Without V8 deser.	0.57	0.57	0.57	ChatGPT loss ^a
Without Snappy ^b	0.33	0.33	0.33	2/3 comp. fail
Without depth track.	1.00	1.00	1.00	No effect ^c
Without TipTap parser	1.00	1.00	1.00	Regex gets drafts ^d
Generic regex only	0.57	0.57	0.57	ChatGPT loss ^a

^aClaude and generic platforms unaffected ($P/R/F1 = 1.00$); average across all 7 cases.

^bSnappy tested on 3 ChatGPT compressed cases only. ^cDepth tracking prevents false positives on deeply nested real-world data. ^dTipTap parser provides deduplication and structure; regex loses these.

Table 9: LevelDB storage statistics across 22 active directories.

Metric	Value
Active directories	22
Total WAL log files	22
Total SSTable files (.ldb)	14
Total WAL size	5.6 MB
Total SSTable size	0.7 MB
Largest single WAL	2,473 KB
Mean WAL size	261 KB

Table 18: Attack surface by platform. “Profiles” shows the number of IndexedDB directories containing LLM data on our test workstations.

Platform	Profiles	Privs	Path
macOS Chrome	23	User	~/Library/ .../Chrome/
macOS Edge	–	User	~/Library/ .../Edge/
macOS Desktop	1	User	~/Library/ .../Claude/
Windows Chrome	1	User	%LOCALAPPDATA%/ .../Chrome/
Windows Edge	0	User	%LOCALAPPDATA%/ .../Edge/
Linux Chrome	†	User	~/config/ google-chrome/
Linux Chromium	†	User	~/config/ chromium/
Linux Brave	†	User	~/config/ BraveSoftware/

† Linux paths validated via simulated filesystem test; no live data collected.

Table 19: Schema-drift robustness (recall of message text, 4-message synthetic conversation). “Struct.” is the structured V8 parser alone; “Full” is the engine including the keyword fallback.

Schema mutation	Struct.	Full	Path
Baseline (observed schema)	1.00	1.00	structured
Reordered keys (text before id)	1.00	1.00	structured
Renamed id (id→uuid)	1.00	1.00	structured
Renamed anchor (messages→mapping)	0.00	1.00	fallback
Added nesting (text one level deeper)	0.00	1.00	fallback
Renamed anchor (kept text field)	0.00	1.00	fallback
Renamed text (text→content)	0.00	0.00	none
Combined (anchor + text renamed)	0.00	0.00	none

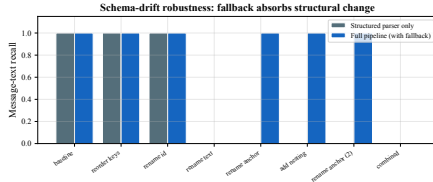


Figure 8: Schema-drift robustness. The structured parser (gray) breaks on anchor renames and added nesting, but the keyword fallback (blue) restores full recall; only renaming the content field itself defeats both—a change that also breaks grep/JSON baselines.

Table 16: Unicode recovery across writing systems and encodings. V8-aware carvers (ChatGPT) and JSON-based carvers (Claude) achieve 100% recovery. The generic regex carver fails on non-ASCII due to JSON Unicode escape sequences.

Script	V8 (0x22)	V8 (0x63)	Claude
English (ASCII)	✓	✓	✓
Emoji	✓	✓	✓
Hindi (Devanagari)	✓	✓	✓
Chinese (CJK)	✓	✓	✓
Japanese	✓	✓	✓
Math symbols	✓	✓	✓
Mixed multilingual	✓	✓	✓
Total	7/7	7/7	7/7

Table 17: Recovery under data corruption (72 cases). The parser degrades gracefully, recovering partial data rather than crashing. Truncation and zero-fill are single-run; byte-flip is mean $\pm$ std over 10 trials.

Type	Level	Rec.	Type	Level	Rec.
Truncation	5% rem.	5/5	Byte-flip	3%	0.6/5
Truncation	10% rem.	4/5	Byte-flip	5%	0.3/5
Truncation	25% rem.	3/5	Byte-flip	10%	0.0/5
Truncation	40% rem.	3/5	Byte-flip	20%	0.0/5
Truncation	50% rem.	2/5	Zero-fill	@ 10%	0/5
Truncation	75% rem.	1/5	Zero-fill	@ 25%	1/5
Truncation	90% rem.	0/5	Zero-fill	@ 50%	2/5
Byte-flip	1%	2.6/5	Zero-fill	@ 75%	4/5
Byte-flip	2%	2.4/5	Zero-fill	@ 90%	4/5

Crash rate: 0/72 (0%) across all corruption tests.

Table 10: Data source breakdown for three representative profiles.

Profile	WAL	SST	WAL %	Total
ChatGPT P5	138	0	100%	138
ChatGPT P2	124	0	100%	124
Claude P29	8	94	8%	102

Schema drift. The V8 carver is tuned to the schema we observed, raising the question of how brittle it is to a platform silently changing that schema. We synthetically mutate the ChatGPT V8 schema and run the *actual* carver code (`experiment_schema_drift.py`). Table 19 reports message-text recall under eight mutations: the carver is fully robust to key reordering and metadata renames; it degrades gracefully to the keyword fallback when the array anchor is renamed; recovery fails only when the content field itself is renamed—a mutation that also defeats grep/JSON baselines.

Role assignment. We quantify the parity heuristic’s fragility using synthetic conversations with known roles (experiment_role_assignment.py). Table 20 shows the heuristic is exact under strict alternation but degrades when broken: a single insertion flips *every* subsequent label. *Text recovery is unaffected in all cases*—only the user/assistant label can be wrong.

Table 20: Role-assignment accuracy under conversation structures that violate strict user/assistant alternation (6-turn synthetic conversations). Text recovery is 100% throughout; only the role label is affected.

Conversation structure	Role accuracy
Strict alternation (user first)	6/6 (1.00)
Two regenerations (even shift)	5/6 (0.83)
One regenerated response	2/6 (0.33)
Interleaved tool/plugin call	1/6 (0.17)
Leading system prompt	0/6 (0.00)
Assistant-first transcript	0/6 (0.00)
Micro-average	14/36 (0.39)

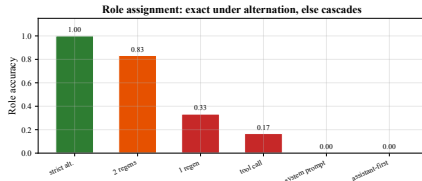


Figure 9: Role-assignment accuracy by conversation structure (6-turn synthetic transcripts). Parity is exact under strict alternation but cascades once an odd number of insertions (system prompt, single regeneration, tool call) flips downstream parity; two regenerations restore it (0.83). Text recovery is unaffected in every case.

M Supplementary Figures

This appendix collects figures referenced from the main text: the tool-comparison bar chart (Figure 10, Section 6.3), the recovered-prompt length distribution (Figure 11, Section 6.7), and the analytical persistence model (Figure 12, Section 6.10), and the artifact-type breakdown (Figure 13, Section 6.9).

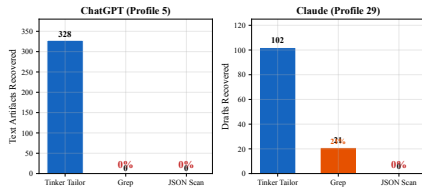


Figure 10: Tool comparison on real LevelDB data. On ChatGPT data, both grep and JSON scan recover 0% of what TINKER TAILOR finds. On Claude data, grep recovers only 21%.

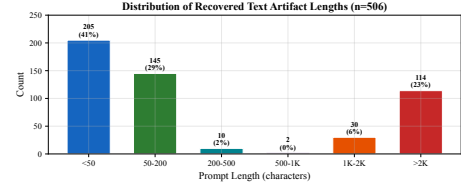


Figure 11: Distribution of recovered prompt lengths. The bi-modal distribution shows short queries (<50 chars, 41%) and substantial content (>2000 chars, 23%), with the latter likely containing code, documents, or detailed technical questions.

Table 21: Analytical persistence model from LevelDB defaults ($M = 4$ MB memtable, $k = 4$ L0 trigger). $T_{\text{flush}} = M/W$ approximates WAL-resident survival time; $T_{\text{reclaim}} = kM/W$ approximates first eligible reclamation of L0 data. The observed 83-day artifact corresponds to an estimated low-activity write rate of $W \approx 49$ KB/day.

Write rate W	T_{flush}	T_{reclaim}
10 KB/day	410 d	1638 d
49 KB/day	83 d	334 d
100 KB/day	41 d	164 d
500 KB/day	8.2 d	33 d
2 MB/day	2.0 d	8.0 d
10 MB/day	0.4 d	1.6 d

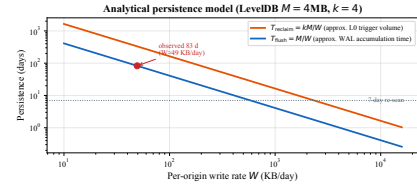


Figure 12: Analytical persistence model versus per-origin write rate ($M = 4$ MB, $k = 4$; log-log). Persistence scales approximately as $\approx 1/W$; the single observed 83-day artifact (red) is consistent with a low-activity write rate of ≈ 49 KB/day. The dotted line marks the 7-day re-scan window.

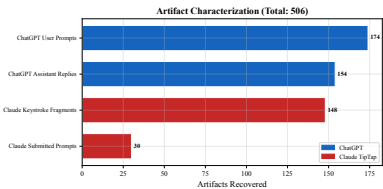


Figure 13: Artifact-type breakdown of the 506 recovered text artifacts across the 23-profile scan. ChatGPT V8 artifacts dominate in volume, while Claude TipTap artifacts comprise substantial submitted/draft prompts (30) and intermediate keystroke fragments (148).